AutoFill Assistant — Implementation Plan

A browser-extension–based assistant that stores a user's personal details and documents **once**, then fills repetitive online forms (government, college, scholarship, exam, certificate portals) on their behalf — pausing for human steps, stopping before submission, and requiring explicit approval.

This document is written to be handed directly to **Claude Code**. Each phase has a goal, a hard "Must Achieve" definition-of-done, "Should Achieve" stretch items, explicit out-of-scope items, and verification steps.

0. Non-negotiable product principles

These are constraints, not preferences. Every phase is checked against them.

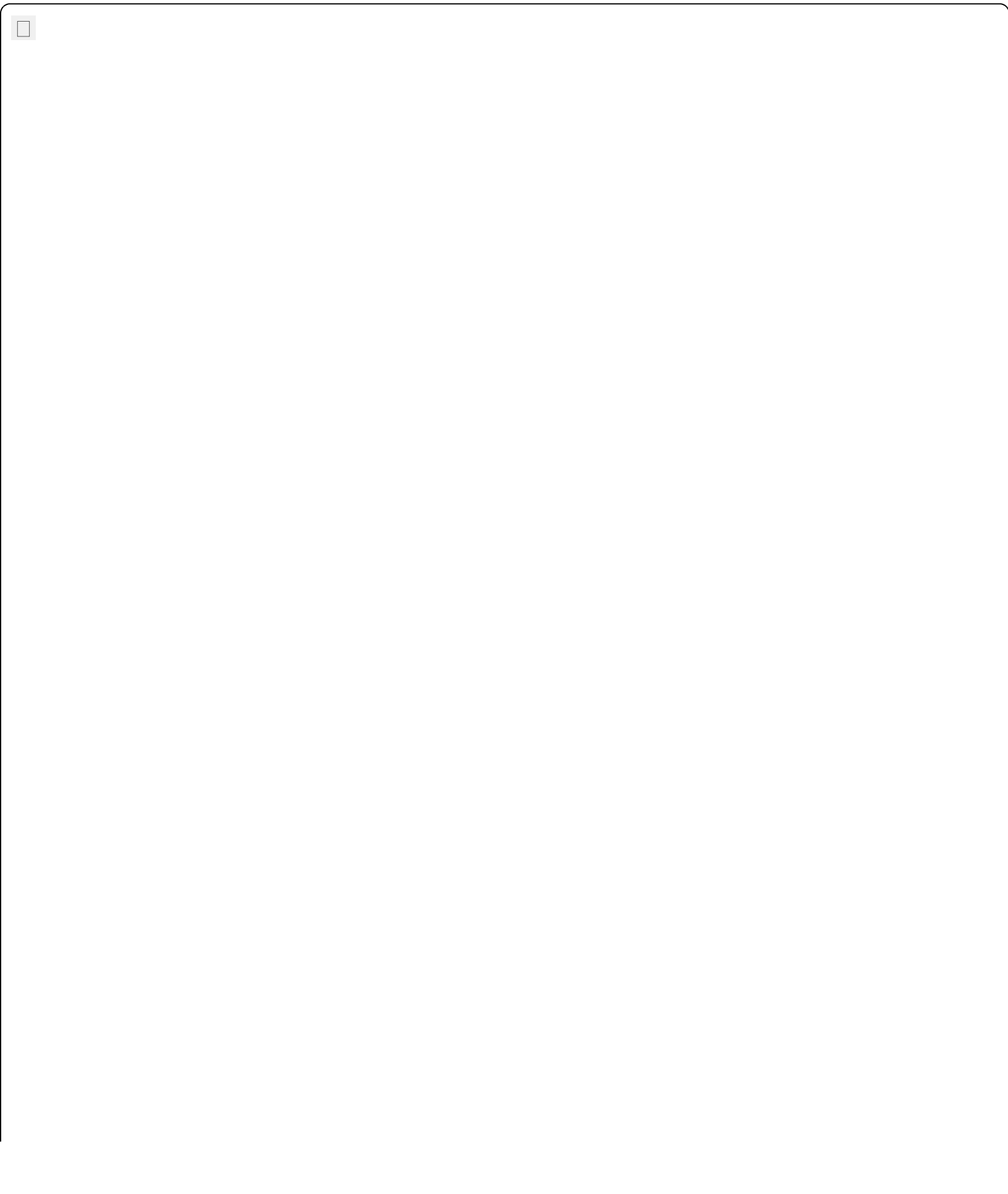
#	Principle	Enforcement
P1	**The user is present.** This assists a human at the keyboard; it is not an unattended bot.	No headless mode, no scheduled runs, no queue of accounts.
P2	**Never auto-submit.** Submission happens only after an explicit approval action on a review screen.	Capture-phase submit interceptor (Phase 5), armed only by an approval token (Phase 7).
P3	**Raw PII never leaves the device.**	LLM receives DOM skeletons and a field-key vocabulary — never values. Enforced by a redaction layer + test suite (Phase 8).
P4	**Never defeat a security control.** No CAPTCHA solving, no OTP interception, no eSign automation.	Those states hand control back to the user (Phase 6).
P5	**Every run is auditable.**	Append-only local audit log: what was filled, from where, what the user approved.
P6	**One account, one user.**	No multi-profile bulk submission features. Ever.

Claude Code: if a requested feature conflicts with P1–P6, stop and flag it rather than implementing it.

1. Technology decisions (locked)

Concern	Choice	Reason
Platform	Chrome Extension, **Manifest V3**	Runs inside the user's own authenticated session. No credential handling, no scraping infrastructure.
Language	TypeScript (strict)	Field mapping is schema-heavy; types catch most mapping bugs.
Build	Vite + `@crxjs/vite-plugin`	HMR for content scripts.
UI	React 18 + Tailwind	Side panel, review screen, vault editor.
Vault storage	IndexedDB via `idb`	Blobs (documents) + structured profile.
Crypto	WebCrypto, AES-GCM 256, key from PBKDF2-SHA256 (≥600k iters)	No external crypto dependency. Argon2id via WASM is a Phase 10 upgrade.
Run state	`chrome.storage.session`	Survives page reloads/postbacks, dies with the browser session.
Image pipeline	`OffscreenCanvas` + `createImageBitmap`	Client-side only; no upload of documents anywhere.
PDF handling	`pdf-lib` (write), `pdf.js` (render→raster)	Certificate merges, size reduction.
State machine	Hand-rolled reducer over a discriminated union	Auditable and serialisable. (XState is acceptable if you prefer; keep transitions declarative either way.)
Testing	Vitest (unit) + Playwright (e2e against fixture pages)	Fixtures are saved real portal HTML.
LLM	Anthropic API, `claude-sonnet-4-6`, called from the background service worker only	Single egress point → easy to audit for PII leaks.

2. Repository layout



```
autofill-assistant/
├── CLAUDE.md                # standing instructions (see §3)
├── IMPLEMENTATION_PLAN.md   # this file
├── manifest.config.ts
├── vite.config.ts
├── src/
│   ├── background/          # service worker: LLM calls, run orchestration
│   │   ├── index.ts
│   │   ├── llm-client.ts
│   │   └── redaction.ts
│   ├── content/             # injected into portal pages
│   │   ├── index.ts
│   │   ├── harvester.ts     # DOM → FieldDescriptor[]
│   │   ├── filler.ts        # write values, dispatch events
│   │   ├── submit-guard.ts  # P2 enforcement
│   │   ├── picker.ts        # teach-mode element picker
│   │   └── overlay/         # in-page status pill, pause banner
│   ├── vault/
│   │   ├── crypto.ts
│   │   ├── db.ts
│   │   ├── profile-store.ts
│   │   └── document-store.ts
│   ├── documents/
│   │   ├── derive.ts        # master → variant
│   │   ├── constraints.ts   # parse "JPEG, 20–50KB, 3.5×4.5cm"
│   │   └── raster.ts
│   ├── matching/
│   │   ├── taxonomy.ts      # canonical field keys (see §4)
│   │   ├── deterministic.ts # rule + score matcher
│   │   ├── profile-cache.ts # site profiles
│   │   └── llm-matcher.ts
│   ├── runtime/
│   │   ├── machine.ts       # fill run state machine (see §5)
│   │   ├── plan.ts          # FillPlan types + fingerprinting
│   │   └── audit.ts
│   ├── ui/
│   │   ├── sidepanel/
│   │   ├── review/
│   │   ├── vault-editor/
│   │   └── onboarding/
```

```
│   └── shared/
│       ├── messages.ts         # typed message bus
│       └── types.ts
├── fixtures/                   # saved portal HTML, one dir per site profile
│   └── <site-id>/page-1.html
├── site-profiles/              # versioned JSON selector maps (shippable)
│   └── <site-id>.json
└── tests/
```

3. `CLAUDE.md` — write this first

Create it before any code. Contents:

```
markdown
# Project: AutoFill Assistant

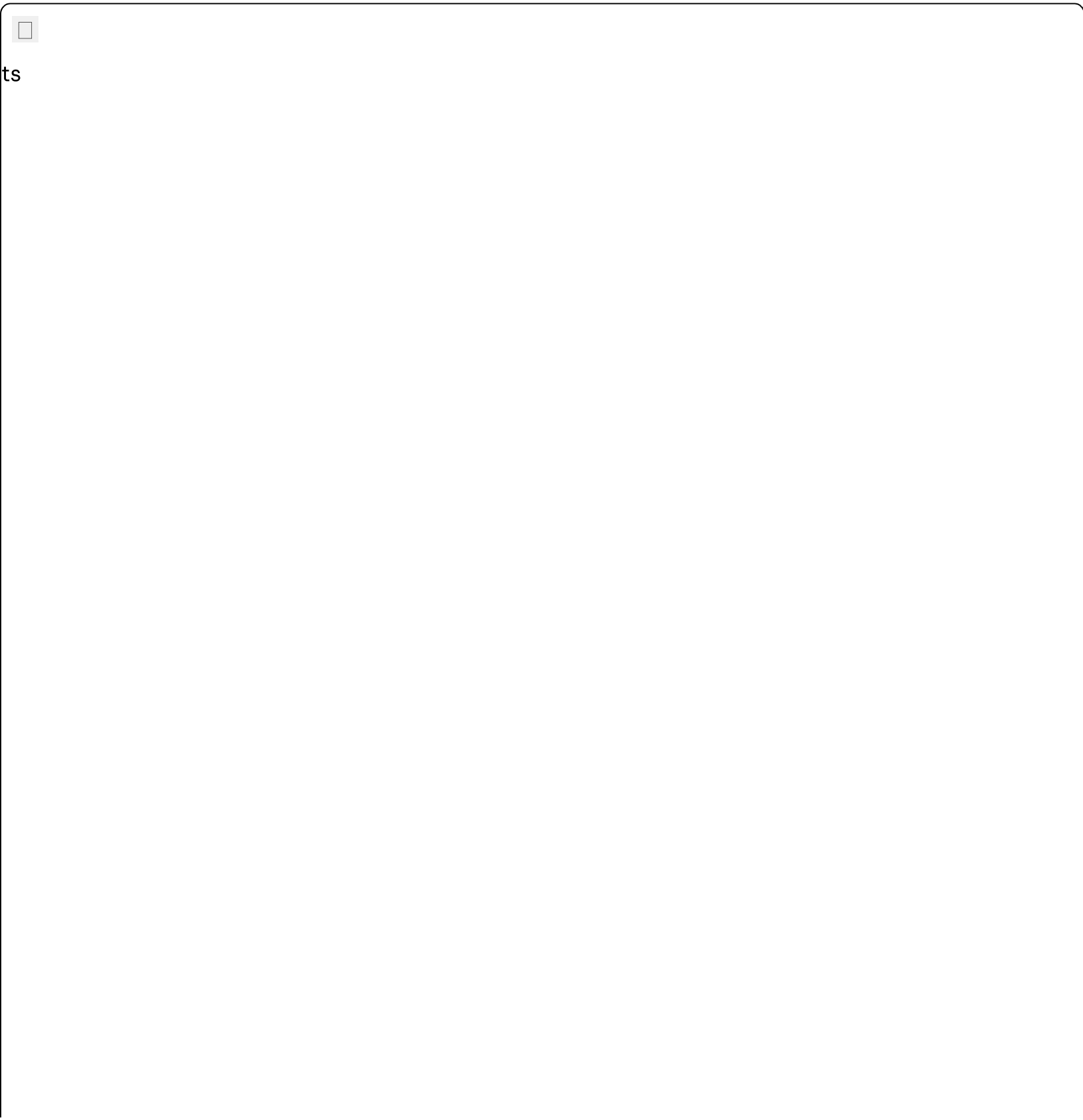
## Hard rules
1. Never write code that clicks a submit/final-action element outside `submit-guard.ts`,
  and only when `approvalToken` is present and valid.
2. Never include user profile values, document bytes, or filenames in any network request.
  All outbound LLM payloads must pass through `redaction.ts` and its test suite.
3. Never implement CAPTCHA solving, OTP reading, or eSign automation.
4. All selectors live in versioned site-profile JSON. No hard-coded selectors in TS.
5. Every new field-matching rule needs a fixture test.

## Working style
- Test-first for `matching/`, `documents/`, `runtime/machine.ts`, `vault/crypto.ts`.
- One phase per branch: `phase/<n>-<slug>`.
- Do not start phase N+1 until phase N's "Must Achieve" checklist is green.
- Prefer small, pure functions in `matching/` and `documents/` — they must be unit-testable
  without a browser.

## Commands
- `npm run dev` — extension dev build
- `npm test` — vitest
- `npm run e2e` — playwright against fixtures
- `npm run fixtures:serve` — serve fixtures/ on :4173
```

4. The canonical field taxonomy (design this before anything else)

Everything — vault schema, matcher output, site profiles, LLM vocabulary, review screen — keys off one stable namespace. Getting this wrong is expensive later.



```ts
// matching/taxonomy.ts
export const FIELD_KEYS = [
  // identity
  'identity.full_name', 'identity.given_name', 'identity.middle_name',
  'identity.surname', 'identity.dob', 'identity.gender', 'identity.category',
  'identity.nationality', 'identity.marital_status', 'identity.religion',
  'identity.mother_tongue', 'identity.blood_group',
  // family
  'family.father_name', 'family.mother_name', 'family.guardian_name',
  'family.father_occupation', 'family.annual_income',
  // contact
  'contact.email', 'contact.mobile', 'contact.alt_mobile', 'contact.landline',
  // address (permanent | correspondence)
  'address.permanent.line1', 'address.permanent.line2', 'address.permanent.city',
  'address.permanent.district', 'address.permanent.state', 'address.permanent.pincode',
  'address.permanent.country',
  'address.correspondence.*',          // same shape
  // government ids  (HIGH SENSITIVITY — see §6.3)
  'id.aadhaar', 'id.pan', 'id.passport', 'id.voter', 'id.driving_licence',
  'id.abc_id', 'id.apaar',
  // education (repeatable rows)
  'education[].level', 'education[].board_or_university', 'education[].institution',
  'education[].roll_number', 'education[].year_of_passing',
  'education[].marks_obtained', 'education[].max_marks', 'education[].percentage',
  'education[].cgpa',
  // banking
  'bank.account_holder', 'bank.account_number', 'bank.ifsc', 'bank.bank_name',
  'bank.branch', 'bank.account_type',
  // documents
  'doc.photo', 'doc.signature', 'doc.aadhaar_card', 'doc.pan_card',
  'doc.marksheet_10', 'doc.marksheet_12', 'doc.degree_certificate',
  'doc.income_certificate', 'doc.caste_certificate', 'doc.domicile_certificate',
  'doc.bank_passbook', 'doc.disability_certificate',
] as const;

export type FieldKey = typeof FIELD_KEYS[number];
```

Rules:

Keys are append-only. Renaming a key breaks every shipped site profile — add a new one and alias.

`sensitivity: 'low' | 'medium' | 'high'` is attached per key; `high` keys require re-authentication before fill (Phase 6).
The LLM is given this list as a closed vocabulary and may only return keys from it, plus `"unknown"`.

5. The fill-run state machine (the hard part — design before Phase 5)

Because CAPTCHA/OTP hand control back to the user, and because ASP.NET postbacks regenerate DOM IDs, the run **cannot** be a single linear pass. Model it explicitly:

```ts
type RunState =
  | { t: 'IDLE' }
  | { t: 'DETECTING' }                       // is there a form here?
  | { t: 'PLANNING'; fields: FieldDescriptor[] }      // match fields → keys
  | { t: 'NEEDS_TEACHING'; unmapped: FieldDescriptor[] }
  | { t: 'READY'; plan: FillPlan }               // user sees "Fill this form"
  | { t: 'UNLOCKING' }                       // vault passphrase / re-auth
  | { t: 'FILLING'; plan: FillPlan; cursor: number }
  | { t: 'BLOCKED'; reason: BlockReason; plan: FillPlan } // captcha | otp | esign | payment | unknown_widget
  | { t: 'REVERIFYING'; plan: FillPlan }          // after DOM churn / postback / resume
  | { t: 'DRIFTED'; plan: FillPlan; lost: FieldRef[] }  // selectors no longer resolve
  | { t: 'REVIEW'; plan: FillPlan; diff: ReviewDiff }
  | { t: 'ARMED'; plan: FillPlan; approvalToken: string; expiresAt: number }
  | { t: 'SUBMITTED'; at: number }
  | { t: 'ABORTED'; why: string }
  | { t: 'ERROR'; err: SerializedError };
```

Transition rules that must be enforced by tests:

`FILLING → BLOCKED` on detecting a CAPTCHA widget, OTP input, or unknown security iframe.
`BLOCKED → REVERIFYING` only on user signal ("I've done it, continue") **or** a MutationObserver settling event — never on a timer alone.
Any `document` navigation, `__doPostBack`, or mutation touching >20% of tracked nodes forces `→ REVERIFYING`.
`REVERIFYING` re-resolves every field ref; if any value was reset → refill it; if a selector fails → `DRIFTED`.
`DRIFTED → NEEDS_TEACHING` (invite repair) — never silently skip a field.
`REVIEW → ARMED` requires an explicit user gesture; `approvalToken` is a random 128-bit value with a **90-second TTL** and is single-use.
`ARMED → SUBMITTED` permits exactly **one** programmatic click on exactly the element recorded in the plan. Token is burned first.

Any state → `ABORTED` on user Esc / panel close / tab close.

Persistence: the entire `RunState` + `FillPlan` is serialised to `chrome.storage.session` under key `run:${tabId}:${formFingerprint}` after every transition. Content script re-hydrates on load. This is what makes pause/resume survive a postback.

Form fingerprint: stable hash of `{origin, pathname, sorted(field name/id/label tokens), form action}` — deliberately excludes volatile ASP.NET `__VIEWSTATE`, GUID-ish ids, and index suffixes.

Phase 0 — Foundations & legal groundwork

Goal: repo, toolchain, target list, and a written boundary of what you will and won't automate.

Must Achieve

- [ ] Repo scaffolded; `npm run dev` loads an unpacked MV3 extension in Chrome with a working side panel showing "Hello".
- [ ] `CLAUDE.md` committed (§3).
- [ ] `matching/taxonomy.ts` committed with the full key list and per-key sensitivity (§4).
- [ ] **Three target portals chosen** and their ToS/acceptable-use pages read and summarised in `docs/portal-notes/<site-id>.md`, each ending with an explicit verdict: `assistive-ok` / `unclear` / `prohibited`. Do not build profiles for `prohibited`.
- [ ] `docs/THREAT_MODEL.md`: what an attacker gets from a stolen laptop, a malicious portal page, a malicious extension update, and a compromised API key.
- [ ] Fixture corpus: for each target, save the real form pages (`Ctrl+S`, complete) into `fixtures/<site-id>/`. Strip any real PII from fixtures.

Should Achieve

- [ ] `npm run fixtures:serve` serves fixtures locally so e2e tests never touch live portals.
- [ ] CI running lint + unit tests.

Out of scope: any real form filling.

Verify: `npm test` passes with zero tests; extension loads; a teammate can read `docs/portal-notes/` and understand which portals are in-bounds.

Phase 1 — The vault

Goal: encrypted local storage for profile data. No forms yet.

Must Achieve

- [ ] `vault/crypto.ts`: AES-GCM 256 encrypt/decrypt; key derived from passphrase via PBKDF2-SHA256 ≥600k iterations, per-vault random salt, per-record random IV. **Unit tested**, including tamper-detection (flipped ciphertext byte → decrypt throws).
- [ ] Master key held only in the service worker's memory, never persisted, never in `chrome.storage.local`.
- [ ] Auto-lock: key zeroed after 15 min idle or on browser close.
- [ ] `vault/profile-store.ts`: CRUD over the taxonomy, with per-key `{value, verifiedAt, source}` provenance.
- [ ] Vault editor UI: user can enter profile data once and see it persisted across a browser restart (re-entering passphrase).
- [ ] Export/import of an encrypted backup file (`.afvault`), with a scary-but-clear warning.

Should Achieve

- [ ] Field-level validation: Aadhaar Verhoeff checksum, PAN regex, IFSC format, PIN code range, DOB sanity.
- [ ] "Completeness meter" — % of taxonomy filled, so users can front-load data entry.
- [ ] Passphrase strength meter; refuse anything under a reasonable floor.

Out of scope: documents (Phase 2), cloud sync (never, or a much later deliberate decision).

Verify: kill the browser, reopen, unlock, data intact. Inspect IndexedDB in DevTools — you must see only ciphertext. Wrong passphrase → clean failure, no partial decrypt.

Phase 2 — Document vault & the derivation pipeline

Goal: store one high-quality master per document; generate portal-specific variants on demand. This is the "upload once" promise and it is where a lot of the real value lives.

Must Achieve

- [ ] Store masters as encrypted blobs with metadata `{key, mime, bytes, width, height, dpi, pages, capturedAt}`.
- [ ] `documents/constraints.ts`: a `DocConstraint` type — `{mime[], minBytes, maxBytes, minW, minH, maxW, maxH, aspect?, dimensionsMm?, dpi?}` — plus a parser that turns portal instruction text ("Photograph must be JPEG/JPG, 20 KB to 50 KB, 3.5 cm × 4.5 cm") into one.
- [ ] `documents/derive.ts`: given master + constraint, produce a compliant file via:
resize to target pixel dimensions (respect mm+DPI when given),
binary-search JPEG quality to land inside the byte window,
pad/letterbox only if aspect is fixed and cropping would cut content,
PDF→JPEG rasterisation and JPEG→PDF wrapping as needed,
fail loudly if the window is unsatisfiable (e.g. 20 KB max at 600×800 from a blurry source) with a human-readable reason.
- [ ] Derived variants cached, keyed by `hash(masterId + constraint)`.
- [ ] Unit tests over a fixture set of ≥10 real-world constraint strings, asserting output mime/bytes/dimensions.

Should Achieve

- [ ] In-browser crop/rotate/deskew for the master.
- [ ] Background removal / white-background normalisation for photos.
- [ ] Signature cleanup (threshold to black-on-white).
- [ ] Quality gate on upload: warn if the master is too small to satisfy common constraints — better to fix at intake than mid-form.

Out of scope: OCR, auto-extracting profile data from documents (tempting; defer to Phase 10+).

Verify: feed a 4 MB phone photo and a "20–50 KB, 3.5×4.5 cm JPEG" constraint; output must be JPEG, within bytes, correct pixel dims, visually acceptable.

Phase 3 — Extension shell & DOM harvester

Goal: on any page, produce a precise structured description of every fillable field. No filling.

Must Achieve

- [ ] Content script injected on user activation (`activeTab` + explicit click), **not** broad `<all_urls>` auto-injection.
- [ ] `content/harvester.ts` emits `FieldDescriptor[]`:

```ts
interface FieldDescriptor {
  ref: string;          // stable-ish internal handle
  selectors: string[];  // ranked: id → name → label-anchored XPath → nth-of-type
  tag: 'input'|'select'|'textarea'|'radio-group'|'checkbox'|'file'|'custom';
  type?: string;
  labelText: string;    // resolved from <label for>, aria-label, wrapping text, table cell to the left
  placeholder?: string;
  nearbyText: string;   // trimmed, for LLM context
  options?: {value:string; text:string}[];
  maxLength?: number; pattern?: string; required: boolean;
  frameId: number;      // iframes are first-class
  visible: boolean; enabled: boolean;
}
```

- [ ] Label resolution handles the ugly cases: `<table>` layouts, `<label>` siblings without `for`, floating labels, `aria-labelledby`, and text nodes preceding the input.
- [ ] Same-origin iframe traversal; cross-origin frames are detected and reported as `BLOCKED:unknown_widget` candidates rather than silently skipped.

- [ ] Side panel shows the harvested list live for the active tab.

Should Achieve

- [ ] Shadow DOM piercing.
- [ ] Detect multi-step wizards (step indicators, `Next` buttons) and record step index in the fingerprint.
- [ ] Detect dependent dropdowns (State → District → Taluk) by observing XHR/mutation after a select changes.

Out of scope: matching, filling.

Verify: run the harvester over every fixture page; snapshot-test the output. Manually confirm ≥95% of visible fields are captured with a non-empty `labelText`.

Phase 4 — Deterministic matcher & site profiles

Goal: map `FieldDescriptor → FieldKey` without any LLM, for the three target portals.

Must Achieve

- [ ] `matching/deterministic.ts`: scored matcher combining
exact/normalised match against a synonym dictionary per `FieldKey` (include Hindi/regional-language labels for government portals),
`autocomplete` attribute mapping (`given-name`, `postal-code`, …),
`name`/`id` token heuristics (`txtFatherName`, `ddlState`, `hdnPan`),
type/`maxLength`/`pattern` compatibility as a *filter* (a 10-char numeric field is not `identity.full_name`),
option-set matching for selects (a select containing all 28 Indian states ⇒ `address.*.state`).
- [ ] Output `MatchResult { fieldRef, key | 'unknown', confidence, rationale, source: 'profile'|'rule'|'llm' }`.
- [ ] **Confidence thresholds:** ≥0.85 auto-fill; 0.5–0.85 fill but flag for review; <0.5 leave blank and mark unmapped.
- [ ] `site-profiles/<site-id>.json` schema, versioned:

```
json
```

```
{
 "siteId": "example-portal",
 "version": 3,
 "matchUrl": "https://portal.example.gov.in/apply*",
 "formFingerprint": "sha256:...",
 "createdAt": "...", "lastVerifiedAt": "...",
 "fields": [
  { "key": "family.father_name", "selectors": ["#txtFName"], "confidence": 1.0, "source": "taught" }
 ],
 "documents": [
  { "key": "doc.photo", "selectors": ["#fuPhoto"],
    "constraint": { "mime": ["image/jpeg"], "minBytes": 20480, "maxBytes": 51200,
                    "dimensionsMm": [35,45] } }
 ],
 "submit": { "selectors": ["#btnFinalSubmit"], "label": "Final Submit" },
 "blockers": [ { "type": "captcha", "selectors": [".g-recaptcha", "#imgCaptcha"] } ]
}
```

- [ ] Profile cache lookup precedes rules; rules precede LLM (Phase 8).

Should Achieve

- [ ] Synonym dictionary loaded from JSON so non-developers can extend it.
- [ ] Auto-detect `address.correspondence` vs `address.permanent` from section headings.
- [ ] Handle "Same as permanent address" checkboxes.

Out of scope: LLM fallback.

Verify: across the three fixture portals, deterministic matching alone maps **≥70%** of fields at ≥0.85 confidence, with **zero** high-confidence *mis*-mappings (a wrong high-confidence match is a bug of a different severity class than a miss — treat it as P0).

Phase 5 — Fill engine & the submit guard

Goal: actually write values into a page, robustly, and make submission structurally impossible without approval.

Must Achieve

- [ ] `content/filler.ts` writes values in a way frameworks accept:
use the native value setter (`Object.getOwnPropertyDescriptor(HTMLInputElement.prototype,'value').set`) so React/Angular register the change,

dispatch `input`, `change`, `blur` (and `keydown/keyup` where a portal listens for them),
selects: match by option value, then exact text, then normalised text, then fuzzy — record which strategy won,
radio/checkbox groups: match by label text,
file inputs: build a `File` from the derived variant, wrap in `DataTransfer`, assign to `input.files`, dispatch `change`.
- [ ] Sequential filling with a small randomised inter-field delay (60–200 ms) — this is for **dependent-dropdown correctness and not tripping naive rate heuristics**, not for deception. Wait for dependent selects to repopulate before filling the child.
- [ ] Per-field verification immediately after write: read back the value; mismatch ⇒ retry once ⇒ mark `fillFailed`.
- [ ] `content/submit-guard.ts`:
installed at `document_start`, capture phase, on `click`, `submit`, and `keydown[Enter]`,
blocks any submit-ish activation unless a valid, unexpired, unburned `approvalToken` is present,
blocks `form.submit()` by patching the prototype for the page's forms (documented as best-effort),
shows an in-page toast when it blocks something, so the behaviour is never mysterious.
- [ ] In-page overlay: a status pill (filling / paused / review ready) the user can always see, plus an **Abort** button.

Should Achieve

- [ ] "Fill this one field" mode from the picker.
- [ ] Undo — restore the pre-fill snapshot of every touched field.
- [ ] Respect `maxLength` and truncate with a review flag rather than silently corrupting data.

Out of scope: review screen, approval.

Verify: e2e on fixtures — every mapped field ends up with the correct value; clicking the real submit button while unarmed does nothing except show the toast. Write an adversarial test where the fixture page calls `form.submit()` on load.

Phase 6 — Pause, resume, and drift recovery

Goal: survive CAPTCHAs, OTPs, and postbacks. This is the hardest phase — budget accordingly.

Must Achieve

- [ ] `runtime/machine.ts` implementing §5 exactly, as a pure reducer, **fully unit-tested** with no DOM.
- [ ] Blocker detection: reCAPTCHA/hCaptcha/Turnstile iframes, `img` elements with `captcha` in src/alt/id, inputs named `otp|token|vcode`, DigiLocker/eSign redirects, payment gateway origins.
- [ ] On block: overlay explains *what* is needed, extension stops touching the page, "I've completed this — continue" button appears.
- [ ] MutationObserver + `webNavigation` listeners force `REVERIFYING` on postback/navigation.

- [ ] `REVERIFYING` re-resolves every selector against the live DOM, re-reads every filled value, refills what was reset, and escalates to `DRIFTED` when a selector no longer resolves.
- [ ] Full run state persisted to `chrome.storage.session` on every transition; content script rehydrates on load and continues without data loss.
- [ ] Re-authentication gate: filling any `sensitivity: 'high'` key (Aadhaar, PAN, bank account) after an idle period requires passphrase re-entry.

Should Achieve

- [ ] Multi-step wizard support: a plan spanning steps, with per-step review checkpoints.
- [ ] Session-expiry detection (redirect to login) → clean `ABORTED` with a "log in and press resume" path.
- [ ] Draft-save detection — prefer the portal's own "Save as draft" over a resume-from-scratch when one exists.

Out of scope: anything that reads, solves, or bypasses a challenge.

Verify: build a fixture that (a) resets all field values on a simulated postback, (b) regenerates element IDs each load, (c) contains a fake CAPTCHA gate. The run must pause, resume after the user "solves" it, detect the reset, refill, and reach `REVIEW` with correct data. Run it 20× — flakiness here is a real defect, not test noise.

Phase 7 — Review screen & approval

Goal: the user's last line of defence. Make it genuinely reviewable, not a rubber stamp.

Must Achieve

- [ ] Review UI (side panel) listing every field: **label as shown on the portal**, value filled, source key, confidence, and match rationale.
- [ ] Visual grouping by form section; click a row → scroll to and highlight the field in the page.
- [ ] Explicit surfacing of: low-confidence matches, truncated values, fuzzy select matches, `fillFailed` fields, and **fields the portal requires that are still empty**.
- [ ] Document section: thumbnail of each derived variant, plus its byte size and dimensions vs. the stated requirement.
- [ ] Inline edit — corrections write back to the page **and** offer "save this correction to my vault".
- [ ] Approval requires a deliberate gesture (type-to-confirm or hold-to-confirm), not a single stray click. Issues a 90-second single-use `approvalToken`.
- [ ] After approval: exactly one click on the recorded submit element, then `SUBMITTED`. Token burned before the click.
- [ ] `runtime/audit.ts`: append-only local log — timestamp, site, fields filled (keys, not values, unless the user opts into value logging), what was flagged, what the user edited, approval time.

Should Achieve

- [ ] Side-by-side diff against the last submission to the same portal.
- [ ] Post-submission capture: store the acknowledgement/reference number and a PDF/screenshot of the confirmation page into the vault, linked to the run.
- [ ] "Explain this match" — show the rationale chain for any field.

Out of scope: LLM matching.

Verify: a user-testing pass — hand it to 3 people with one deliberately wrong value planted. All three should catch it. If they don't, the review screen has failed its only job; redesign it before proceeding.

Phase 8 — LLM fallback matcher

Goal: generalise to portals with no site profile — **without** sending PII anywhere.

Must Achieve

- [ ] `background/redaction.ts`: builds the LLM payload from `FieldDescriptor`s only — label text, name/id, type, options, nearby text. Vault values, document bytes, and filenames are structurally unreachable from this code path (enforce with a module boundary + lint rule).
- [ ] **Leak test suite:** seed the vault with canary strings; run the matcher over all fixtures; assert no canary appears in any outbound payload. This test is mandatory in CI.
- [ ] Prompt contract — model returns strict JSON, closed vocabulary:

```
You map form fields to a fixed vocabulary of profile keys.

ALLOWED KEYS: <FIELD_KEYS list>
Return "unknown" if no key fits. Never invent keys.

FIELDS:
[{"ref":"f12","label":"पिता का नाम / Father's Name","name":"txtFName",
  "type":"text","maxLength":50,"nearby":"Personal Details"}, ...]

Return ONLY a JSON array:
[{"ref":"f12","key":"family.father_name","confidence":0.0-1.0,"why":"<12 words"}]
```

- [ ] Response validation: unknown keys rejected, confidence clamped, refs matched back; malformed JSON → one retry → fall back to `unknown`.
- [ ] Results merged **under** deterministic results (rules always win ties) and capped at 0.8 confidence — an LLM guess never auto-fills a `sensitivity: 'high'` field without user confirmation.

- [ ] Successful runs are written into a new site profile so the LLM is called **once per form, ever**.
- [ ] Graceful degradation: no API key / offline / rate-limited ⇒ deterministic-only mode with a clear notice.

Should Achieve

- [ ] Batch fields into one call; cache by DOM-skeleton hash.
- [ ] Ask the model to extract document constraints from instruction text on the page (feeds Phase 2's parser).
- [ ] Token/cost meter in settings.

Out of scope: sending any user value to any model, for any reason, including "verification".

Verify: on a *fourth* portal with no profile, LLM + rules maps ≥85% of fields correctly. Canary test green. Delete the API key → extension still works on profiled sites.

Phase 9 — Teach mode & profile repair

Goal: turn your maintenance burden into a user-driven feature. This is what makes the product survive portal redesigns.

Must Achieve

- [ ] `content/picker.ts`: element picker overlay — user clicks a field, chooses a `FieldKey` from a searchable list, mapping is recorded.
- [ ] Entered from two places: `NEEDS_TEACHING` (unmapped fields) and `DRIFTED` (broken selectors).
- [ ] Taught mappings write a new site-profile **version**; old versions retained for rollback.
- [ ] Selector recording generates a ranked, resilient list (id → name → label-anchored XPath → structural path), and prefers selectors that survive across your fixture's regenerated-ID variants.
- [ ] Health check: on load, if <60% of a profile's selectors resolve, mark it stale and prompt for repair rather than filling a half-broken form.
- [ ] Document-slot teaching: user points at a file input, picks a document key, and either types the constraint or lets the parser read it from the page.

Should Achieve

- [ ] Export/import site profiles as JSON so users can share them (community profile repo).
- [ ] If you ever host a shared profile registry: profiles contain **selectors only, never values**; review submissions before publishing; sign them. Treat an untrusted profile as untrusted input.
- [ ] Telemetry (strictly opt-in, aggregate only): which profiles are drifting, so you know what to fix.

Out of scope: automatic profile publishing without review.

Verify: take a fixture, rename every `id`, reload. The extension must detect drift, enter teach mode, and after ~2 minutes of user clicking, fill correctly again — with a bumped profile version.

Phase 10 — Hardening, packaging, release

Goal: something you can responsibly give to strangers.

Must Achieve

- [ ] Strict CSP; no `eval`; no remote code. Audit every dependency (`npm audit`, licence check).
- [ ] Content-script isolation review: the page must not be able to reach vault data, the approval token, or the message bus. Treat every portal page as hostile.
- [ ] Message-bus hardening: validate sender, reject messages from page context, schema-validate every message.
- [ ] Upgrade KDF to Argon2id (WASM) with migration from PBKDF2 vaults.
- [ ] Rate/scope limits enforcing P1 and P6: one active run per tab, hard cap on runs per hour, no headless entry point.
- [ ] Onboarding flow: vault creation → guided profile entry → document upload with quality checks → first supervised fill on a demo fixture form.
- [ ] Clear in-product disclosure: what the extension does, what leaves the device, what the user is responsible for verifying, and that they remain responsible for the accuracy of anything submitted.
- [ ] Privacy policy + Chrome Web Store listing that honestly matches behaviour (store review will check permission justification — `activeTab`, `storage`, `sidePanel`, `scripting`, host permissions per profile).
- [ ] Crash-safe: killing the browser mid-run leaves no half-filled state that resumes incorrectly.

Should Achieve

- [ ] Firefox port (MV3 differences in service workers / `browser.*` namespace).
- [ ] Accessibility pass on all extension UI (this is an accessibility product for many users — screen-reader support is not optional in spirit).
- [ ] Localisation of the extension UI (Hindi + one regional language to start).
- [ ] Optional encrypted backup to user-chosen storage, key never leaving the device.

Verify: external security review of `vault/`, `submit-guard.ts`, and `redaction.ts` at minimum. Full e2e suite green 20 consecutive runs.

6. Cross-cutting requirements

6.1 Testing strategy

Unit (Vitest): crypto, taxonomy, deterministic matcher, constraint parser, image derivation, state machine reducer. No DOM.
Fixture snapshot: harvester output per fixture page, committed and diffed.

E2E (Playwright): full runs against served fixtures, including hostile variants — regenerated IDs, postback resets, fake CAPTCHA, `form.submit()` on load, cross-origin iframe.
Leak tests: canary values in vault; assert absence in all outbound payloads. **CI-blocking.**
Manual portal smoke tests: before each release, one real run per supported portal, stopped at review, never submitted (use a scratch/dummy application where the portal permits one).

6.2 Definition of "done" for any phase

Must-Achieve checklist fully green.
Tests written *and* failing-before/passing-after for every new rule.
No new hard-coded selectors outside `site-profiles/`.
Leak test suite green.
`docs/` updated — especially portal notes and the threat model.

6.3 Sensitivity tiers

Tier	Examples	Rule
high	Aadhaar, PAN, bank account, passport	Re-auth before fill; never auto-filled from an LLM-only match; masked in the review UI until revealed; excluded from any log by default.
medium	DOB, address, mobile, marks	Normal fill; shown in review.
low	Name, gender, state	Normal fill.

6.4 Risk register

Risk	Severity	Mitigation
High-confidence **mis**-match submits wrong data	Critical	Rationale in review UI; type-to-confirm; misc-match treated as P0 defect class; zero-mismatch gate in Phase 4.
Portal ToS prohibits automation	High	Phase 0 verdicts; assistive-with-human-approval framing; drop `prohibited` portals.
Vault compromise on stolen device	High	Strong KDF, memory-only key, auto-lock, high-tier masking.
Portal redesign breaks everything	Medium	Versioned profiles + drift detection + teach mode (Phase 9).

Risk	Severity	Mitigation
Store rejection over permissions	Medium	`activeTab` + per-profile host permissions, never `<all_urls>`; honest listing.
LLM cost blowup	Low	Profile caching means one call per form lifetime.

6.5 Suggested sequencing

Phases 0→7 are the true MVP and are strictly ordered. Phase 8 (LLM) and Phase 9 (teach mode) can run in parallel afterwards. **Do not build Phase 8 first** — a generic matcher with no working fill/review loop is the classic way to spend six months and fill zero forms.

7. First three Claude Code sessions

Session 1 — scaffold

Read IMPLEMENTATION_PLAN.md and CLAUDE.md. Execute Phase 0 only. Scaffold the MV3 + Vite + TS + React extension, create `matching/taxonomy.ts` with the full key list and sensitivity tiers from §4, set up Vitest and Playwright, and add `npm run fixtures:serve`. Stop and show me the Phase 0 checklist status. Do not write any matching or filling code.

Session 2 — vault

Execute Phase 1. Test-first: write `vault/crypto.test.ts` before `vault/crypto.ts`, including a tamper-detection test. Then the profile store and the vault editor UI. Report against the Phase 1 Must Achieve checklist.

Session 3 — documents

Execute Phase 2. Start with `documents/constraints.ts` and its parser tests using these 10 real constraint strings: [paste them]. Then `derive.ts` with the binary-search quality loop. Prove it with a test that takes `fixtures/photo-4mb.jpg` to a 20–50 KB, 3.5×4.5 cm JPEG.

Then continue one phase per session, checking the gate each time.